EE 446
Saesha Wadhwa
Lab 6 Report

Part 1: Edge Impulse Integration and Deployment

Link to Edge Impulse project: https://studio.edgeimpulse.com/public/989430/live

1) Raw inputs: State the raw inputs used in the Edge Impulse project.
The raw inputs were time-series accelerometer measurements collected from the onboard accelerometer sensor for fan activity detection.

2) NN classifier features: State the type of features passed into the NN classifier, the number of input features, and at least five actual feature names used to train the classifier.
Spectral features generated from the accelerometer vibration were passed into the NN classifier. There were 33 input features. Some of them in order of decreasing importance are as follows: accX RMS, accX Peak 1 Height, accY Peak 3 Height, accX Peak 1 Freq, accY RMS, accY Peak 2 Freq, accY Peak 1 Height ,accZ Peak 2 Height, accZ Peak 3 Height, accZ Peak 3 Freq.

3) NN classifier outputs: State the outputs of the NN classifier.
There were two outputs of the NN classifier which were normal activity and anomaly activity.

4) Anomaly detector features: State the type of features passed into the anomaly detector, the number of features used, and the feature names used to train the anomaly detector.
Spectral features of the accelerometer vibration reading data were passed into the anomaly detector. 4 features were used in this detection. They were accX Peak 1 Height, accX RMS, accY Spectral Power, accZ Spectral Power.

5) Deployment evidence: Include a screenshot showing the Edge Impulse model running on the Arduino Nano 33 BLE Sense and producing inference results.

```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 49 ms, inference 615 us, anomaly 519 us, postprocess
ing 49 us
#Classification predictions:
  nominal: 0.945312
  off: 0.054687
Anomaly prediction: 400.838287
```

```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 49 ms, inference 566 us, anomaly 562 us, postprocess
ing 49 us
#Classification predictions:
  nominal: 0.968750
  off: 0.031250
Anomaly prediction: 12.188734
```

```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 593 us, anomaly 551 us, postprocess
ing 49 us
#Classification predictions:
  nominal: 0.027344
  off: 0.972656
Anomaly prediction: 9.507980
```

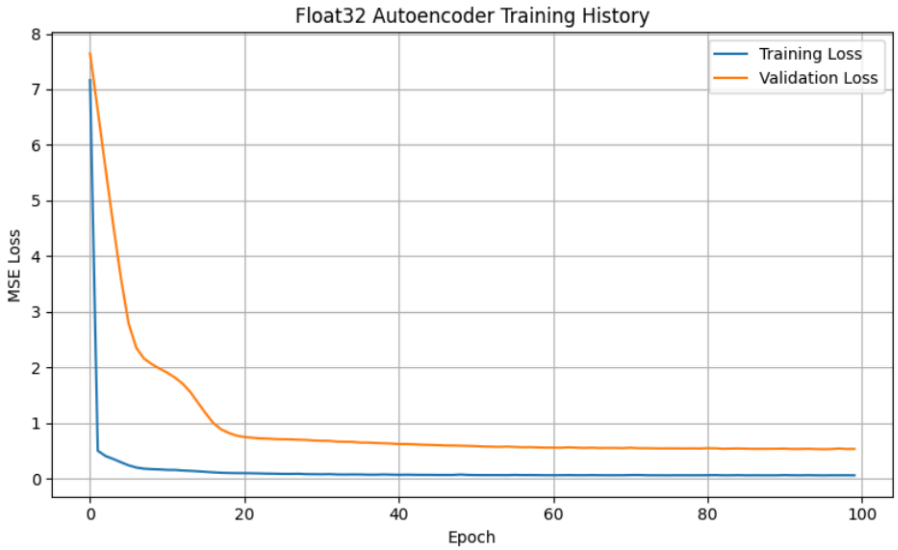
6) Short interpretation: Briefly interpret the observed deployment results. Discuss whether the classifier output should always be trusted and under what conditions it may or may not be reliable

The deployment results were as I expected. When I did large motions like horizontal, vertical, or circular sweeps,they were flagged as anomalies, while constrained high-speed, low-displacement motion mimicking fan vibration was classified as normal. However, the classifier output should not always be trusted. The raw accelerometer values fluctuate a lot and we were given data for a specific fan under specific conditions, so the model does not generalize well.The classifier was most reliable when deployment conditions closely match exact vibration conditions, and less reliable when tested with modified vibration patterns, noise, or environmental differences that might not have been included in training.

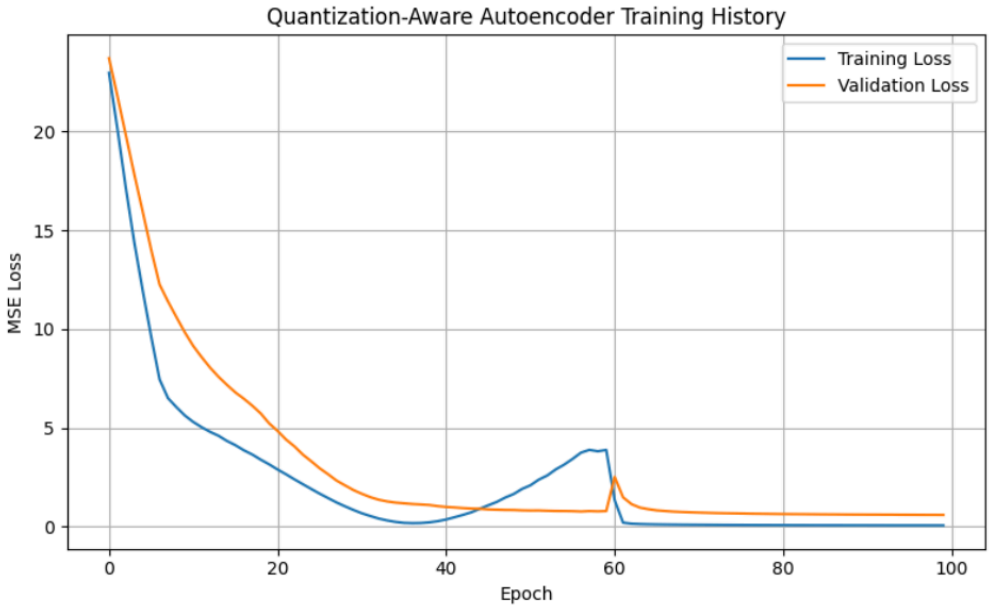
Part 2: Quantized Autoencoder-Based Activity Anomaly Detection

1) Training and validation loss curves

Without Quantisation Aware Training

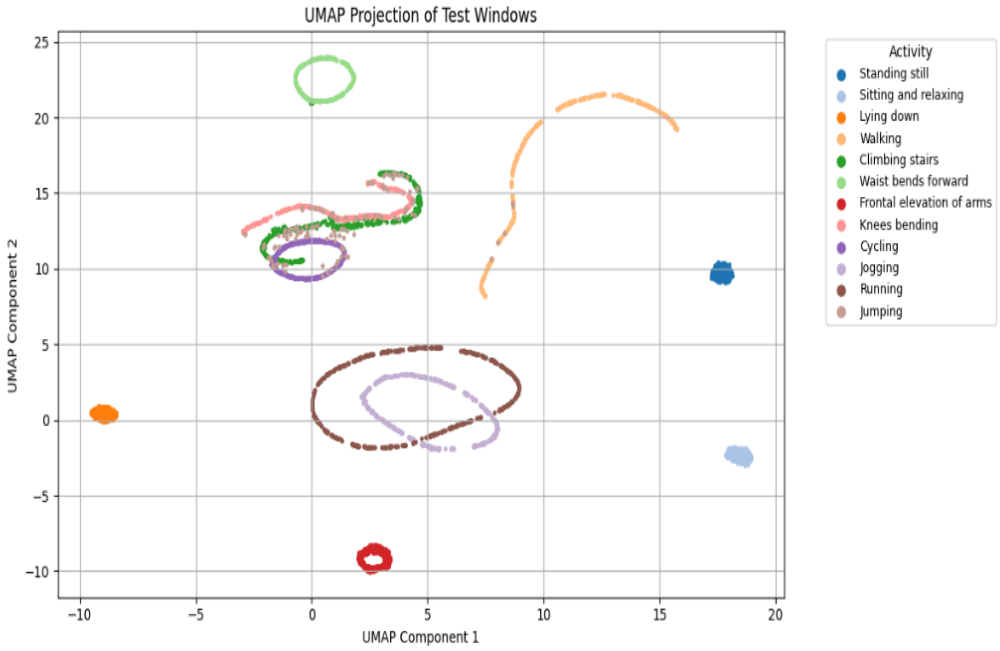


With Quantisation Aware Training



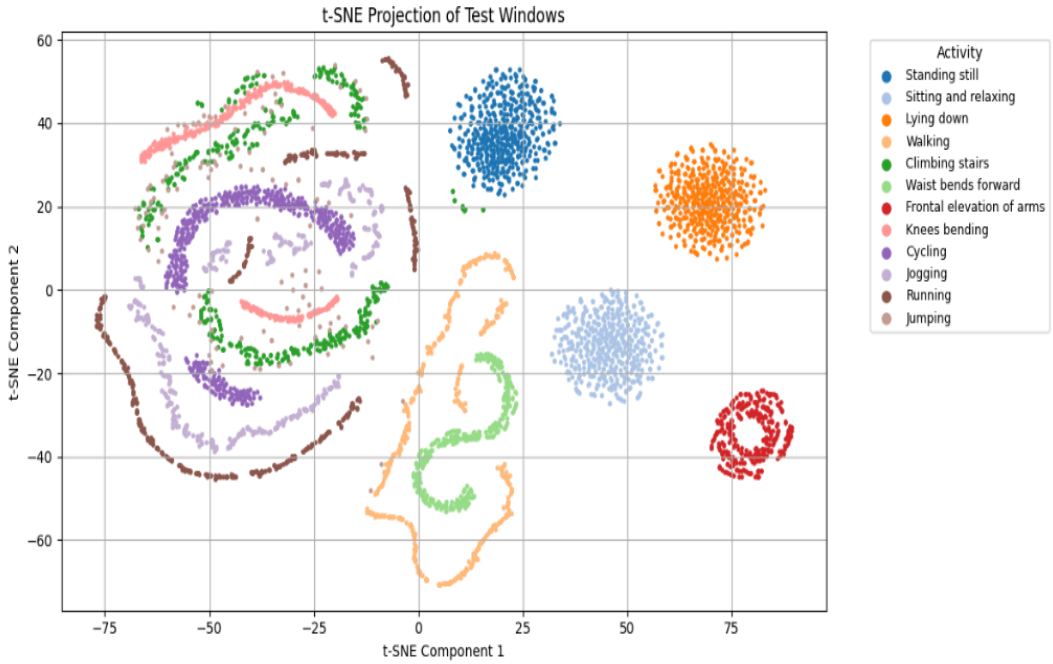
2) PCA, t-SNE, and UMAP visualizations

UMAP Visualization

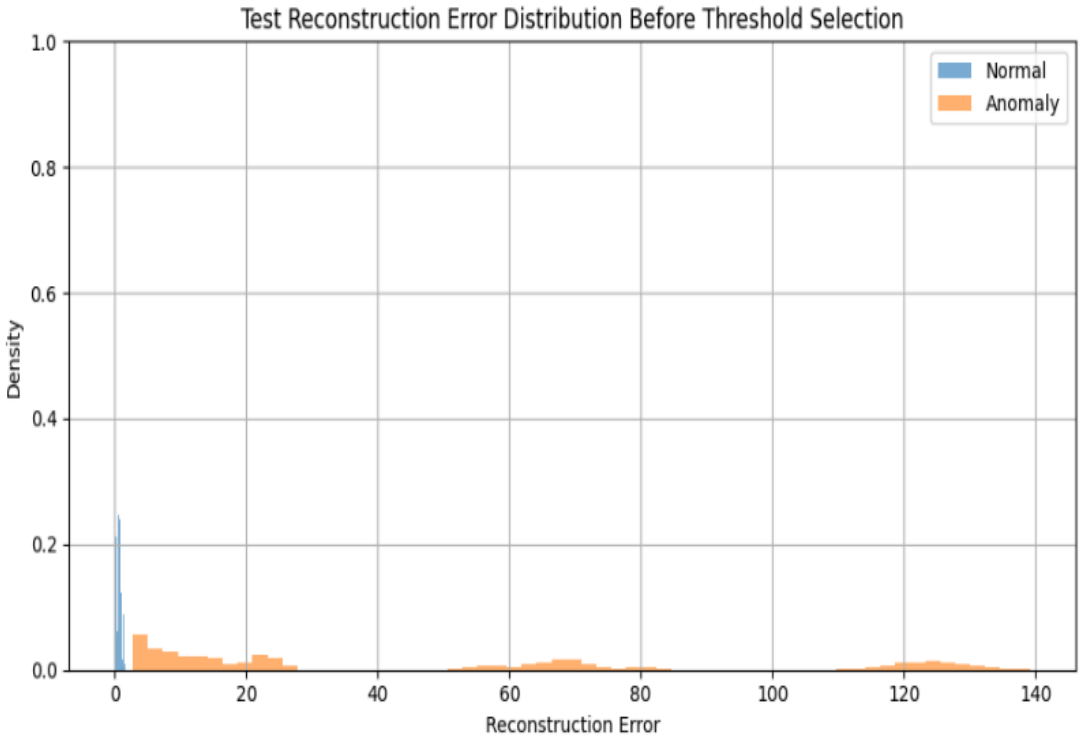


PCA Visualization

t-SNE Visualization



3) Reconstruction error distribution and selected threshold



4) Confusion matrix and classification report

Classification Report

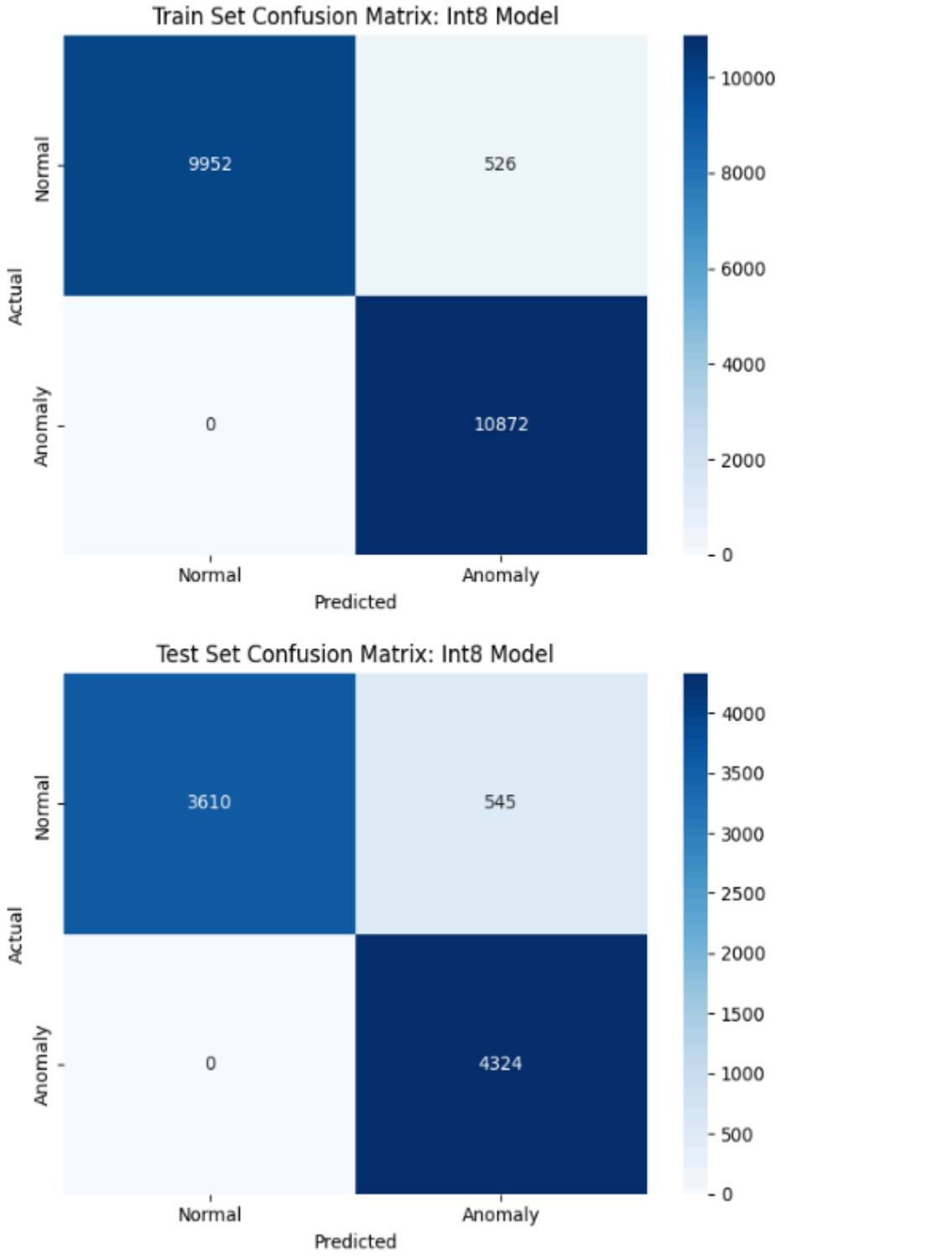
Train set evaluation: int8 model

	precision	recall	f1-score	support
Normal (0)	1.00	0.95	0.97	10478
Anomaly (1)	0.95	1.00	0.98	10872
accuracy			0.98	21350
macro avg	0.98	0.97	0.98	21350
weighted avg	0.98	0.98	0.98	21350

Test set evaluation: int8 model

	precision	recall	f1-score	support
Normal (0)	1.00	0.87	0.93	4155
Anomaly (1)	0.89	1.00	0.94	4324
accuracy			0.94	8479
macro avg	0.94	0.93	0.94	8479
weighted avg	0.94	0.94	0.94	8479

Confusion matrix on test and train set



5) Arduino serial monitor output showing reconstruction error and normal/anomaly detection result

Anomaly Activity

```
Reconstruction error: 2.850868
Result: Anomaly detected
Reconstruction error: 2.850657
Result: Anomaly detected
Reconstruction error: 2.850521
Result: Anomaly detected
Reconstruction error: 2.850273
Result: Anomaly detected
Reconstruction error: 2.850064
Result: Anomaly detected
Reconstruction error: 2.849685
Result: Anomaly detected
Reconstruction error: 2.849419
Result: Anomaly detected
Reconstruction error: 2.849127
Result: Anomaly detected
Reconstruction error: 2.848992
Result: Anomaly detected
```

Normal Activity

```
Result: Normal activity
Reconstruction error: 0.611569
Result: Normal activity
Reconstruction error: 0.611014
Result: Normal activity
Reconstruction error: 0.640871
Result: Normal activity
Reconstruction error: 0.632931
Result: Normal activity
Reconstruction error: 0.600105
Result: Normal activity
Reconstruction error: 0.621652
Result: Normal activity
Reconstruction error: 0.576584
Result: Normal activity
Reconstruction error: 0.563714
Result: Normal activity
Reconstruction error: 0.574857
Result: Normal activity
Reconstruction error: 0.567103
```

6) Short interpretation of the observed results
Normal activities like no movement and precisely still position of the board gave normal inference and error of 0.57 as on these values the encoder trained well. Any slight disturbance or high intensity movement was recorded as anomalous and gave a big reconstruction error of around 2.58 hinting that these movements were not trained as well.

7) Discussion Questions

1)What threshold did you choose for anomaly detection, and why?
I chose a threshold of 0.5739 which is calculated as the 95th percentile of training normal reconstruction errors. I checked the mean and max of normal errors and the mean and min of anomaly errors, there is a large gap between the two with no overlap. Since normal data has low reconstruction error and anomaly data has high reconstruction error, this threshold separates the two well, classifying anything below it as normal and anything above it as anomalous.

2)How does reconstruction error help distinguish normal and anomalous activity?
The autoencoder is trained only on normal data, so it learns to reconstruct normal windows accurately, producing low error. When an anomalous window is passed through, the model has never seen that pattern before and reconstructs it poorly, producing high error. This difference in reconstruction error, this threshold is what separates normal from anomalous activity, normal windows fall below the threshold and anomalous windows fall above it.

3) What information from the Python notebook must be preserved when deploying the model to Arduino?
The Arduino sketch must preserve the trained model weights, which are saved in the INT8 quantized model. The quantization scale and zero-point must also be preserved since the model expects INT8 inputs and the raw float sensor values from the left ankle X, Y, and Z accelerometer axes must be quantized the same way. The threshold of 0.5739 must be hardcoded in the sketch to make the normal vs anomaly decision. The model size must also fit within the Arduino's available flash and RAM, which is why INT8 quantization was applied, to reduce the model to a deployable size.

4)Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?
The model was trained on specific preprocessing assumptions, like accelerometer X, Y, and Z axes, 100-sample windows, and no activity column removed. If the Arduino reads

from different sensors, uses a different window size, the input will not match what the model learned during training and the reconstruction errors will be meaningless, producing wrong classifications.

5)What are the main limitations of this anomaly detection method when deployed on a microcontroller?

The model also cannot retrain itself on the Arduino, so it cannot retrain to adapt to new users or changes in movement. Microcontrollers have limited memory and power, so inference is slower and the model is small after quantization and has reduced accuracy. Real sensor readings were noisier than the dataset used for training, which could make reconstruction errors higher and makes the threshold less reliable.